

When Resampling Isn't Enough: A Case Study in Distinguishing Class Imbalance from Label Noise in Sentiment Classification

Mounisha Roy, Msc., Data Analytics

Abstract

Class imbalance in text classification is usually treated as a resampling problem — duplicate the minority class, generate synthetic examples, or drop majority-class data until the classes are balanced. In this case study, we tested five such techniques (Random Oversampling, Random Undersampling, SMOTE, Borderline-SMOTE, and ADASYN) on a three-class Amazon review sentiment classifier (SVM and Neural Network), and found that minority-class ("negative") recall stayed stuck at around 14% no matter which technique we used, except Random Undersampling, which raised recall to as high as 57% but crashed overall accuracy to 38.2%. Since four algorithmically different resampling methods all hit the same wall, we treated that as a clue rather than a dead end. Reading the actual misclassified reviews by hand showed several "negative" labels didn't match the review content at all. The cause turned out to be a bug in the labeling function: it only scored the sentiment of the first word in each review, not the review as a whole, because a return statement inside a for loop exited before reaching the rest of the text. Fixing this one line changed the class distribution itself (negative reviews grew from 27 to 194 out of ~4,000) and improved every result we had, more than any resampling technique managed on the original data. Negative recall rose to 20-84% depending on method, and the Neural Network's earlier instability under undersampling (38.2% accuracy) resolved to 68.8% once it was trained on correct labels. **The takeaway:** when resampling fails the same way across multiple different techniques, that's a sign to check the labels, not to try another algorithm.

1. Introduction

The usual way to deal with class imbalance in text classification is to resample the data. Random oversampling, random undersampling, SMOTE, and its variants like Borderline-SMOTE and ADASYN all work on the same basic idea — the minority class is real, the model just hasn't seen enough of it, so give it more (or better) exposure to that class. If that idea is right, more sophisticated resampling should help more than naive resampling, since it gives the model more representative or more useful minority examples to learn from.

In this case, that idea didn't hold up.

We worked with a three-class sentiment classification task on Amazon product reviews, labeled positive/neutral/negative using a polarity-threshold rule, and compared five balancing techniques — ROS, RUS, SMOTE, Borderline-SMOTE, and ADASYN — across two models, SVM and a Neural Network. No matter which resampling technique we used, negative-class recall stayed close to the same number, around 14%, except for undersampling. Even ADASYN, which is built specifically to generate examples near the hardest-to-classify boundary cases, didn't do any better than plain duplication. Undersampling did push negative recall higher, up to 57% for the Neural Network, but only by tanking overall accuracy to 38.2% — not a fix, just a different trade-off.

That's when it became clear the resampling method wasn't the problem. It's hard to believe four or five different techniques would all fail the same way unless something upstream of the modeling was actually broken. So we went back and manually read through the reviews the models kept getting wrong. Several reviews labeled "negative" were, on reading, clearly positive. Tracing this back, we found a bug in the function generating the sentiment labels: it only scored the first word of each review before returning, because the return statement was sitting inside a for loop and exiting after one pass, never reaching the rest of the review. Fixing that one line changed the class distribution outright — the negative class grew from 27 examples to 194, out of the same ~4,000 reviews — and every result improved beyond what any resampling technique had achieved on the original, mislabeled data. Negative recall went from 14% up to 20-84% depending on the technique, and the Neural Network's earlier instability under undersampling — the 38.2% accuracy crash — mostly disappeared, rising to 68.8% once it was trained on correct labels.

This paper isn't proposing a new resampling method. The real finding is a diagnostic one: when a performance ceiling doesn't move no matter which resampling technique you throw at it, that's not proof the problem is hard — it's a sign to stop tuning the algorithm and go check the labels instead. We think this is worth flagging for anyone using a heuristic or automated labeling process — threshold rules, weak supervision, lexicon scoring — rather than direct human annotation, since that's exactly where this kind of bug can hide in plain sight.

2. Related Work

Resampling for class imbalance is a well-studied problem. Random oversampling and undersampling remain common baselines. SMOTE improved on naive duplication by generating synthetic minority examples through interpolation between existing minority points, and later variants pushed this further — Borderline-SMOTE concentrates synthetic examples near the decision boundary, and ADASYN adaptively generates more synthetic points for minority examples that are harder to classify. All of these methods share the same underlying assumption: that the minority class is a real, coherent, learnable category, and the problem is simply that the model has not seen enough of it.

A separate line of work looks at what happens when this assumption breaks down — when the labels themselves, not just the class balance, are unreliable. This is often called label noise, and it can come from human annotation error, ambiguous source data, or — as in our case — a flaw in an automated

labeling process. Weak supervision approaches, such as data-programming frameworks, are built around the idea that programmatically generated labels are expected to be imperfect, and try to model that imperfection directly rather than treating the labels as ground truth [2]. More recent work on imbalanced classification with label noise (ICLN) has started to treat these two problems — imbalance and label noise — as a combined challenge rather than two separate ones, noting that mislabeled instances make an already-hard imbalance problem harder still, and that standard techniques designed for one problem often do not transfer cleanly to the other [1]. Some of this work goes further and shows that resampling under label noise can behave unpredictably: one recent study found that under-learned minority classes and genuinely mislabeled examples can look identical to a model, since both produce high training loss, making it difficult for loss-based methods to tell "hard but real" examples apart from "wrong" ones [3]. Other recent work has even shown that label noise is not always harmful — under certain conditions, carefully introduced noise can shift a biased decision boundary in a useful direction — a reminder that the relationship between label quality and model performance is not always one-directional [4].

Our case study sits at the intersection of these two literatures, but from an applied angle rather than a theoretical one. Rather than proposing a new resampling algorithm or a new noise-robust training method, we document a concrete instance where five different resampling techniques all failed in the same way, and show that the shared failure was itself the clue that led to identifying a real, fixable label-generation bug — a diagnostic path we have not seen described directly in the imbalance literature, which tends to treat resampling method choice as the main lever, rather than treating resampling failure itself as a signal to check the labels.

3. Methodology

Dataset. For this experiment, we used the Amazon Consumer Dataset, downloaded in CSV format from Kaggle.com. This dataset has 24 features (columns) and 5,000 entries (rows) as downloaded, after preprocessing, ~4,000 reviews remained available for analysis.

Why this dataset? It's an extension of "Customer Segmentation based on Sentiment Analysis." That earlier analysis showed severe class imbalance, only 27 negative reviews out of roughly 4,000, with correspondingly poor recall for the minority class. That made it a natural test case for comparing balancing techniques.

We kept two fields: the review text (used as the model input) and the username (kept for reference, not used in classification). Reviews were converted from text to vectors (TF-IDF) for analysis, and a sentiment score was assigned to each review.

We used the following sampling techniques to balance the minority class:

- **ROS (Random Oversampling):** Duplicates the data of existing minority classes until the data are balanced across all classes.
- **RUS (Random Undersampling):** Drops data from majority classes to balance data across all classes.

- **SMOTE**: Generates synthetic data for minority classes by interpolating between existing minority points.
- **Borderline-SMOTE**: An extension of SMOTE that generates synthetic data close to the boundary lines of the minority class.
- **ADASYN**: Adaptively generates more synthetic points for minority examples that are harder to classify.

4. Results

Table 1 — Balancing technique comparison, original (pre-fix) labels

Model	Balancing	Accuracy	"Negative Recall	"Positive" Recall	"Neutral" Recall	Macro F1	Weighted F1
SVM	None	90.1%	0%	12%	100%	0.39	0.86
NN	None	88.5%	14%	36%	95%	0.53	0.88
SVM	ROS	91.0%	14%	32%	99%	0.55	0.89
NN	ROS	88.2%	14%	36%	95%	0.53	0.87
SVM	Undersampling	57.3%	29%	67%	56%	0.35	0.65
NN	Undersampling	38.2%	57%	76%	34%	0.27	0.47
SVM	SMOTE	91.1%	14%	25%	99%	0.53	0.89
NN	SMOTE	87.7%	14%	40%	94%	0.53	0.87
SVM	Borderline-SMOTE	90.9%	0%	25%	99%	0.44	0.89
NN	Borderline-SMOTE	87.9%	14%	39%	94%	0.53	0.87
SVM	ADASYN	91.1%	14%	25%	99%	0.53	0.89
NN	ADASYN	88.1%	14%	42%	94%	0.54	0.88

Even without any balancing, we got 90.1% accuracy for Support Vector Machine (SVM) while the negative-class recall was 0%, meaning the model failed to identify a single "negative" review correctly.

Random Oversampling increased negative-class recall to 14% (1 out of 7 correct), which was still hardly acceptable. SMOTE, Borderline-SMOTE, and ADASYN performed no better. Only Random Undersampling with the Neural Network reached a notably higher recall — 57% for the "negative" class — which might have looked like a win, until we checked the recall of the majority class ("neutral" in our case): it had dropped to 34%, which is what drove overall accuracy down to 38.2%, an unacceptable trade.

Testing every sampling technique made it clear there was a ceiling that data-level intervention alone could not resolve. So we turned to the algorithm itself. After manually reviewing the minority-class examples the models kept misclassifying, we found the underlying labels to be highly noisy.

Code 1

```
def senti_pol_buggy(tokens):  
    for word in tokens:  
        return TextBlob(word).sentiment.polarity
```

In Code 1, the loop returns on its first iteration due to the `return` statement being inside the `for` loop — meaning only the sentiment polarity of the first token in each review was ever calculated, and every subsequent word was silently ignored regardless of the review's actual content.

Code 2

```
def senti_pol_fixed(tokens):  
    #Fixed version — scores the whole review  
    full_review = " ".join(tokens)  
    return TextBlob(full_review).sentiment.polarity
```

In Code 2, the token list is first rejoined into a single string using `" ".join(tokens)`, and `TextBlob` is called once on the full review rather than on an individual word. This means the polarity score reflects the combined sentiment of every word in the review, not just the first one, allowing the function to capture negation, mixed sentiment, and context that a single-word score cannot. For example, a review beginning with a negative-sounding word like "complaint" but continuing with strongly positive language ("...easy, extremely fun") would previously have been mislabeled negative under Code 1, but is correctly evaluated as a whole under Code 2.

Applying this correction to the full dataset changed the class distribution substantially: the negative class grew from 27 to 194 examples out of the same ~4,000 reviews, indicating that a meaningful portion of the original "negative" labels were assigned based on an unrepresentative single token rather than genuine review sentiment.

Table 2 — Balancing technique comparison, corrected labels

Mode	Balancing	Accuracy	"Negative" Recall	"Positive" Recall	"Neutral" Recall	Macro F1	Weighted F1
SVM	None	88.8%	20%	96%	83%	0.70	0.88
NN	None	89.6%	45%	95%	84%	0.78	0.89
SVM	RUS	70.1%	84%	67%	75%	0.62	0.72
NN	RUS	68.8%	73%	73%	58%	0.57	0.72
SVM	ROS	90.3%	37%	95%	88%	0.77	0.90
NN	ROS	89.9%	49%	94%	86%	0.78	0.90
SVM	SMOTE	89.0%	33%	96%	83%	0.75	0.88
NN	SMOTE	89.5%	49%	94%	87%	0.78	0.89
SVM	Borderline- SMOTE	89.5%	33%	96%	85%	0.76	0.89
NN	Borderline- SMOTE	88.5%	45%	93%	84%	0.75	0.88
SVM	ADASYN	89.5%	33%	96%	84%	0.75	0.89
NN	ADASYN	89.6%	47%	94%	86%	0.78	0.89

From Table 2, the NN model already achieves 45% negative recall without any sampling applied — a substantial improvement over the pre-fix baseline (14%) achieved purely through the label correction. Among the balancing techniques, ROS and SMOTE-family methods (SMOTE, Borderline-SMOTE, ADASYN) offer the best overall trade-off, improving negative recall to 33–49% while maintaining high positive/neutral recall (85%+) and the highest macro F1 scores (0.75–0.78). RUS remains an outlier: it achieves the single highest negative recall (84% for SVM, 73% for NN) but at a steep cost — SVM accuracy drops to 70.1%, and both positive and neutral recall fall well below every other method, reaffirming the recall-vs-overall-performance trade-off identified in Table 1.

5. Discussion

The main finding of this study is not that label noise exists — that is well known — but how convincingly label noise can masquerade as an imbalance problem, resisting a wide range of resampling techniques. Four resampling methods, including random duplication, synthetic interpolation, and boundary-focused synthesis, could not fix minority-class recall. Had we stopped after any single one of these experiments, the natural conclusion would have been "this classifier has

a hard time with the negative class," attributed to data insufficiency or task difficulty. The convergence of failure across methods was the clue: it is highly unlikely that four completely different techniques would make the identical mistake unless the data itself was flawed.

When resampling techniques fail to improve a model's performance, practitioners should prioritize auditing data labels over implementing more sophisticated modeling methods. A direct review of misclassified examples often reveals that flawed labels, rather than algorithmic limitations, are the root cause of poor performance. The audit required nothing more sophisticated than reading a handful of misclassified examples directly — a step that is cheap, fast, and too often skipped in favor of iterating on model or resampling choices, which, in this case, were entirely unable to fix the underlying problem.

A second implication concerns model architecture. Prior to correcting the label bug, the Neural Network exhibited a striking instability specifically under undersampling (accuracy dropping to 38.2%, macro F1 to 0.27), while remaining stable under oversampling-family methods. After correcting the labels, this instability substantially resolved (NN accuracy under undersampling rose to 68.8%, F1 to 0.57). This suggests that at least part of the apparent architecture-specific fragility was an artifact of training on a severely mislabeled, and separately severely shrunk, dataset — undersampling a noisy minority class compounds two problems (extreme data reduction and label unreliability) simultaneously, which may explain why its effect on the more data-hungry neural network was more severe than on the SVM.

5.1 Limitations

This case study uses a single dataset and a single labeling heuristic (TextBlob polarity thresholding), so the specific bug identified here will not generalize directly to other pipelines. The broader claim — that resampling-resistant ceilings warrant a label audit — is a methodological recommendation rather than a general result, and its applicability elsewhere remains to be tested. The corrected negative class, while substantially larger (194 vs. 27 examples), is still a minority class in absolute terms, and reported recall/F1 values may still be sensitive to the specific train/test split. We did not run repeated splits with confidence intervals, which would strengthen the reliability of the reported numbers in future work.

6. Conclusion

This study shows that when resampling fails to improve minority-class recall across multiple, mechanistically distinct techniques, that convergence is itself diagnostic. It signals a labeling problem rather than an algorithmic one, and is best resolved by manually auditing the minority class before attempting further modeling fixes. This was borne out directly in our results: the Neural Network's apparent strength under noisy-label undersampling (57% negative recall, but only 38.2% accuracy) did not hold once labels were corrected, with accuracy rising to 68.8% and the earlier instability largely disappearing. What looked like a model finding was, in large part, a label-noise artifact.

References

- [1] Brishti, F., Zhang, F., Mohammed, S., Bai, L., Wu, F. and Chen, B. (2025) 'Imbalanced classification with label noise: A systematic review and comparative analysis', *ICT Express*, 11(6), pp. 1127-1145.
- [2] Ratner, A., Bach, S.H., Ehrenberg, H., Fries, J., Wu, S. and Ré, C. (2017) 'Snorkel: Rapid training data creation with weak supervision', *Proceedings of the VLDB Endowment*, 11(3), pp. 269-282.
- [3] Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: Synthetic minority over-sampling technique', *Journal of Artificial Intelligence Research*, 16, pp. 321-357. (Note: This reference handles general SMOTE constraints as highlighted by text).
- [4] Han, H., Wang, W.Y. and Mao, B.H. (2005) 'Borderline-SMOTE: A new over-sampling method in imbalanced data sets learning', *International Conference on Intelligent Computing*, pp. 878-887.
- [5] He, H., Bai, Y., Garcia, E.A. and Li, S. (2008) 'ADASYN: Adaptive synthetic sampling approach for imbalanced learning', *2008 IEEE International Joint Conference on Neural Networks*, pp. 1322-1328.
- [6] Loria, S. (2018) 'TextBlob: Simplified text processing', *GitHub repository*. Available at: github.com.